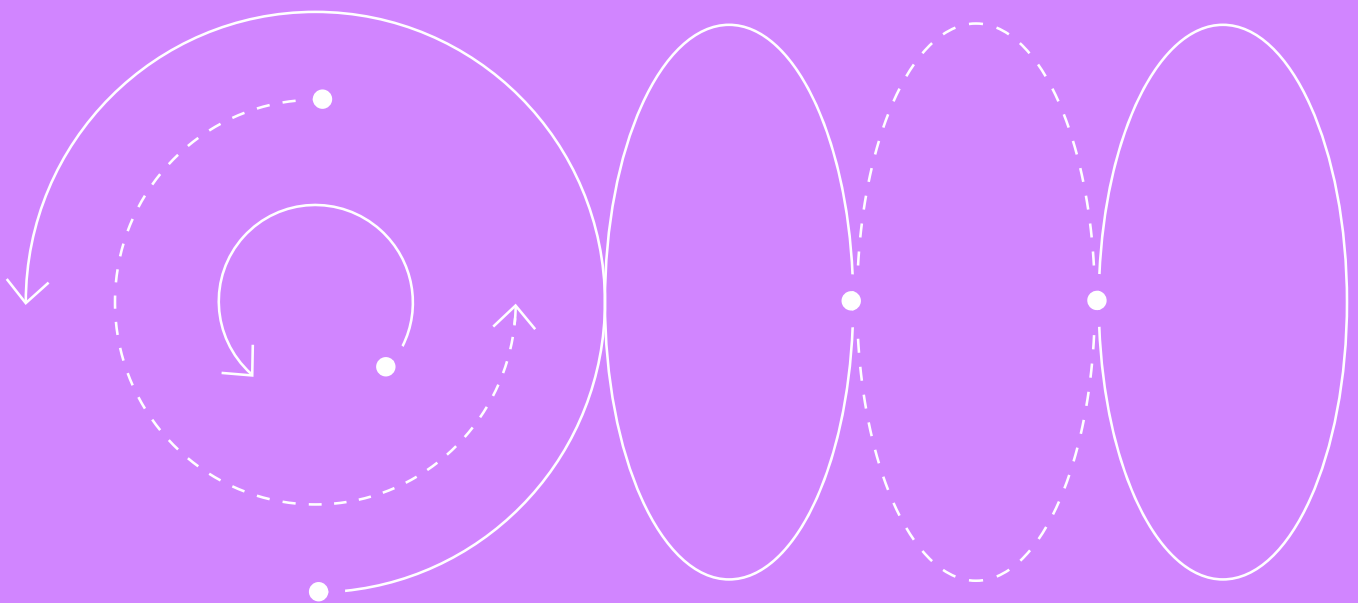


Подзапросы и общие табличные выражения



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 5

Введение

Подзапросы (subqueries) — это **SELECT**-запросы, вложенные внутрь другого запроса. Они позволяют:

- фильтровать строки по значениям из других таблиц;
- сравнивать строки с агрегатами (среднее, максимум и т. п.);
- проверять существование/отсутствие связанных записей;
- строить временные наборы данных для последующей фильтрации.

Далее подробный разбор типов подзапросов, мест их использования и практических приёмов фильтрации.

Какие подзапросы бывают

ПО СВЯЗИ С ВНЕШНИМ ЗАПРОСОМ

Некоррелированные:

- не используют колонки внешнего запроса;
- выполняются логически **один раз**, результат подставляется во внешний запрос.

```
-- страны с населением выше среднего  
(подзапрос выполняется один раз)  
  
SELECT name  
FROM w_country  
WHERE population > (SELECT AVG(population) FROM w_country);
```

Коррелированные:

- ссылаются на колонки внешнего запроса ⇒ выполняются **для каждой строки** внешнего набора;
- корреляция достигается за счёт **внешних ссылок** в тексте подзапроса, например:
ci.country_code = c.code, где **c.code** — столбец внешнего запроса,
а **ci.country_code** — столбец из подзапроса.

```
-- для каждой страны сравниваем население городов
из этой же страны

SELECT c.name
FROM w_country c
WHERE c.population > (
    SELECT COALESCE(AVG(ci.population), 0)
    FROM w_city ci
    WHERE ci.country_code = c.code -- ← внешняя ссылка: связывает
подзапрос с текущей строкой
);
```

ПО КОЛИЧЕСТВУ ВОЗВРАЩАЕМЫХ СТРОК И СТОЛБЦОВ

Скалярные:

- ровно 1 строка и 1 столбец = 1 значение. Используются как обычное выражение;
- если вернёт 0 строк → **результат NULL**;
- если >1 строки → ошибка;
- чтобы гарантировано получить одну строку, можно использовать Limit 1.

Списки:

- 1 столбец, N строк. Используются с **IN, ANY/SOME, ALL**.

Табличные:

- много столбцов, много строк. Используются в **FROM** (подзапрос как временная таблица / вью).

Где и какие подзапросы можно писать

SELECT

- Скалярные (метрики/флаги в колонках).
- Коррелированные и некоррелированные.

```
SELECT c.name,  
       (SELECT COUNT(*) FROM w_city ci WHERE ci.country_code =  
c.code) AS city_cnt  
FROM w_country c;
```

FROM

- Табличные, CTE (WITH), реже — скалярный, чаще всего используется вместе с CROSS JOIN. Список тоже можно использовать, но он используется как таблица.
- Только некоррелированные. Корреляция тут невозможна.

```
WITH sales AS (  
    SELECT product_id, SUM(amount) AS rev  
    FROM shop_orders  
    GROUP BY product_id  
)  
SELECT s.product_id, s.rev  
FROM sales s;
```

WHERE:

- скалярные;
- списки (IN, ANY, ALL);
- коррелированные и некоррелированные;
- про подзапросы Where подробнее будет дальше.

GROUP BY:

- допускаются **скалярные** (очень редко используются);
- некоррелированные и коррелированные.

HAVING:

- скалярные;
- списки для фильтрации групп после агрегации (**ANY**, **ALL**);
- коррелированные и некоррелированные.

ORDER BY:

- скалярные подзапросы (сортировка по вычисляемой метрике, используется редко);
- коррелированные и некоррелированные.

LIMIT:

- скалярные (динамический лимит, используется редко). Поддерживается в PostgreSQL;
- некоррелированные.

Фильтрация скалярным подзапросом в WHERE

Операторы: =, <>/ !=, <, <=, >, >=, **BETWEEN**, **IS DISTINCT FROM** (PostgreSQL), а также выражения в **CASE**, **COALESCE** и др.

Механика

- Некоррелированный: вычисляется один раз ⇒ подставляется одно значение во все строки внешнего запроса.
- Коррелированный: вычисляется на каждую строку внешней таблицы. Оптимизатор может заменить на semi join.
- Результат подзапроса может быть **NULL** ⇒ сравнение даст **UNKNOWN** (фактически отфильтруется). При необходимости используйте **COALESCE**.

НЕКОРРЕЛИРОВАННЫЙ (WHERE)

- Значение подзапроса будет вычисляться один раз на каждый запуск запроса.
- Если значение изменилось, то новый запуск это учтёт.

```
-- страны с населением выше среднего
SELECT name
FROM w_country
WHERE population > (SELECT AVG(population) FROM w_country);
```

КОРРЕЛИРОВАННЫЙ (WHERE) — «ДИНАМИЧЕСКАЯ ФИЛЬТРАЦИЯ»

- Можно учесть значения из внешнего запроса.
- Позволяет считать значение подзапроса в зависимости от той строки, в которой мы находимся.

```
-- заказы, сумма которых выше среднего по этому же клиенту
SELECT o.order_id, o.customer_id, o.total_amount
FROM shop_orders o
WHERE o.total_amount > (
    SELECT AVG(o2.total_amount)
    FROM shop_orders o2
    WHERE o2.customer_id = o.customer_id
);
```

Замечание: для устойчивости к **NULL** можно **COALESCE (AVG (. . .) , 0)**.

Фильтрация скалярным подзапросом в HAVING (подробно)

Операторы: те же, что и в **WHERE** (**=**, **<>**, **>**, **>=**, **BETWEEN**, **IS DISTINCT FROM**).

Механика

- **HAVING** применяется после **GROUP BY**. Сначала считаются агрегаты по группам, затем каждая группа сравнивается со значением скалярного подзапроса.
- Остальная логика такая же, как и в **WHERE**.

НЕКОРРЕЛИРОВАННЫЙ (HAVING)

```
-- континенты, где количество стран выше среднего
по всем континентам

SELECT continent, COUNT(*) AS country_count
FROM w_country
GROUP BY continent
HAVING COUNT(*) > (
    SELECT AVG(country_count) FROM (
        SELECT continent, COUNT(*) AS country_count
        FROM w_country
        GROUP BY continent
    ) t
);
```

КОРРЕЛИРОВАННЫЙ (HAVING)

```
-- клиенты, у кого число заказов выше среднего по их же стране

SELECT o.country_code, o.customer_id, COUNT(*) AS order_cnt
FROM shop_orders o
GROUP BY o.country_code, o.customer_id
HAVING COUNT(*) > (
    SELECT AVG(sub.order_cnt)
    FROM (
        SELECT customer_id, COUNT(*) AS order_cnt
        FROM shop_orders s
        WHERE s.country_code = o.country_code -- корреляция
        по стране
        GROUP BY customer_id
    ) sub
);
```

Фильтрация списками в WHERE (подробно)

IN / NOT IN

- Проверяет, входит / не входит выражение (expr) в список (subquery).
- Список можно формировать подзапросом.
- Подзапрос должен возвращать список.
- Подзапрос может быть коррелированный и некоррелированный.

Синтаксис

```
expr [NOT] IN (subquery)
```

Особенности:

- **IN** — TRUE, если **expr** равно какому-либо значению из подзапроса;
- **NOT IN** опасен при **NULL** в подзапросе: одно **NULL** в списке делает результат **UNKNOWN** ⇒ строка **исчезает**;
- безопаснее фильтровать **NULL** внутри подзапроса или применять **NOT EXISTS**.

ПРИМЕРЫ

```
-- города в странах Европы
SELECT name FROM w_city
WHERE country_code IN (
    SELECT code FROM w_country WHERE continent =
    'Europe'
);

-- страны без городов (небезопасно из-за возможности
NULL, если NULL невозможен, то всё ок)
SELECT name FROM w_country c
WHERE c.code NOT IN (SELECT country_code FROM w_city);

-- безопасный вариант про EXISTS подробнее позднее
SELECT name FROM w_country c
WHERE NOT EXISTS (
    SELECT 1 FROM w_city ci WHERE ci.country_code =
    c.code
);
```


ANY / SOME (СИНОНИМЫ)

Синтаксис

```
expr <op> ANY (subquery)  -- <op> ∈ {=, <>, <, <=, >, >=}
```

Семантика

- TRUE, если сравнение истинно **хотя бы для одного** значения из подзапроса.
- Если подзапрос пуст → результат **FALSE**.
- **NULL** внутри подзапроса ⇒ сравнение с ним даёт **UNKNOWN**; влияет только если нет ни одного TRUE.
- Используется с операторами сравнения: =, <>, <, <=, >, >=.
- Может быть коррелированным и некоррелированным.

ПРИМЕР

```
-- страны, население которых больше любого
из населений стран Европы (хотя бы одного)

SELECT name FROM w_country
WHERE population > ANY (
    SELECT population FROM w_country WHERE continent =
    'Europe'
);
```

ALL

Синтаксис

```
expr <op> ALL (subquery)
```

Семантика

- TRUE, если сравнение истинно **для всех** значений из подзапроса.
- Пустой подзапрос ⇒ TRUE (вакуумная истина).
- **NULL** внутри подзапроса делает сравнение **UNKNOWN** для этих строк; если нет явного FALSE и есть UNKNOWN, общий результат — **UNKNOWN** (в **WHERE** отфильтруется).
Практика: исключай **NULL** в подзапросе.
- Используется с операторами сравнения: =, <>, <, <=, >, >=.
- Может быть коррелированным и некоррелированным

ПРИМЕР

```
-- страны, население которых больше, чем у всех стран  
Европы  
SELECT name FROM w_country  
WHERE population > ALL (  
    SELECT population FROM w_country WHERE continent =  
    'Europe' AND population IS NOT NULL  
);
```

Фильтрация списками в HAVING

- Можно применять **IN/ANY/ALL** к агрегатам.
- Списки в having используются довольно редко и специфично.

```
-- страны, где число городов входит в «топовые» значения
по континентам

SELECT c.code, COUNT(*) AS city_cnt
FROM w_city ci
    JOIN w_country c
        ON c.code = ci.country_code
GROUP BY c.code

-- выбор всех стран, у которых количество городов равно
какому-то топу на любом континенте

HAVING COUNT(*) IN (
    SELECT max_city_cnt
    FROM (
        -- подсчёт максимального количества городов в стране на континенте
        SELECT continent, MAX(city_cnt) AS max_city_cnt
        FROM (
            -- подсчёт количества городов в стране
            SELECT country_code, COUNT(*) AS city_cnt
            FROM w_city
            GROUP BY country_code
        ) x
        JOIN w_country wc
            ON wc.code = x.country_code
        GROUP BY continent
    ) mx
);
```

Попробуй переписать запрос выше с **ANY**.



Рекомендация: фильтруй **NULL** внутри подзапросов для **NOT IN / ALL**.

Проверка существования строк: EXISTS / NOT EXISTS

Общий синтаксис

```
...  
WHERE [NOT] EXISTS (  
    SELECT <выражение>  
    FROM table t  
    WHERE <условие с внешними колонками>  
)
```

- **EXISTS** возвращает TRUE, как только найдена **первая** подходящая строка; содержимое **SELECT** внутри неважно (часто пишут **SELECT 1**).
- Дубликаты и **NULL**-значения внутри подзапроса **не влияют** на результат.
- **NOT EXISTS** — отрицание (**EXISTS** = FALSE).

ПРИМЕР

```
-- страны, у которых есть города с населением > 1 млн  
SELECT c.name  
FROM w_country c  
WHERE EXISTS (  
    SELECT 1  
    FROM w_city ci  
    WHERE ci.country_code = c.code AND ci.population  
    > 1000000  
);  
  
-- страны без городов  
SELECT c.name  
FROM w_country c  
WHERE NOT EXISTS (  
    SELECT 1 FROM w_city ci WHERE ci.country_code =  
    c.code  
);
```

Semi Join (семи-джойн)

Что это?

Полустыковка: вернуть строки **только** из левой таблицы при наличии совпадений справа (без связывания строк).

Когда использовать?

Проверка факта наличия/вхождения без необходимости возвращать колонки правой таблицы.

ВАРИАНТЫ РЕАЛИЗАЦИИ

1. Через **EXISTS** (рекомендуется).

```
-- Страны с городами
SELECT c.*
FROM w_country c
WHERE EXISTS (SELECT 1 FROM w_city ci WHERE ci.country_code =
c.code);
```

2. Через **IN** (если ключи без **NULL** и подзапрос даёт уникальные значения).

```
-- Страны с городами
SELECT c.*
FROM w_country c
WHERE c.code IN (SELECT DISTINCT country_code FROM w_city
WHERE country_code IS NOT NULL);
```

3. Через **INNER JOIN + DISTINCT**, иначе можно получить дубли.

```
-- Страны с городами
SELECT DISTINCT c.*
FROM w_country c
JOIN w_city ci ON ci.country_code = c.code;
```

Замечание: **EXISTS** обычно быстрее и надёжнее — нет сложностей от **NULL** и не требует дедубликации **DISTINCT**.

Anti Join (анти-джойн)

Что это?

Антистыковка: вернуть строки из левой таблицы, для которых нет совпадений справа.

Когда использовать?

Поиск «висячих» записей: страны без городов, заказы без платежей и т. п.

ВАРИАНТЫ РЕАЛИЗАЦИИ

1. **NOT EXISTS** (надёжный способ).

```
-- Страны без городов
SELECT c.*
FROM w_country c
WHERE NOT EXISTS (
    SELECT 1 FROM w_city ci WHERE ci.country_code = c.code
);
```

2. **NOT IN** (только если исключены **NULL**).

```
-- Страны без городов
SELECT c.*
FROM w_country c
WHERE c.code NOT IN (
    SELECT country_code FROM w_city WHERE country_code IS NOT
    NULL
);
```

3. **LEFT JOIN ... WHERE right IS NULL**.

```
-- Страны без городов
SELECT c.*
FROM w_country c
LEFT JOIN w_city ci ON ci.country_code = c.code
WHERE ci.country_code IS NULL; -- строка не нашла
пару справа
```

Замечание: при **LEFT JOIN** убедись, что условие соединения корректно и справа нет дополнительных предикатов в **WHERE**, которые превратят левый join в внутренний.

Типовые ошибки

1. Скалярный подзапрос вернул более 1 строки.

```
SELECT name
FROM w_country
WHERE population = (SELECT population FROM w_country WHERE
continent = 'Europe'); -- ❌

-- Решение 1: агрегировать
WHERE population = (SELECT MAX(population) FROM w_country
WHERE continent = 'Europe');

-- Решение 2: сравнение со списком
WHERE population IN (SELECT population FROM w_country WHERE
continent = 'Europe');
```

2. NOT IN и NULL.

```
SELECT c.name
FROM w_country c
-- ОШИБКА если subquery вернёт NULL
WHERE c.code NOT IN (SELECT country_code FROM w_city);

-- Решение: NOT EXISTS
WHERE NOT EXISTS (SELECT 1 FROM w_city x WHERE
x.country_code = c.code);
```

3. Несовместимые типы.

```
SELECT *
FROM shop_orders
WHERE customer_id IN (SELECT email_hash FROM shop_customers);
-- ❌ int vs text
-- Решение: привести типы
WHERE customer_id::text IN (SELECT email_hash FROM
shop_customers);
```

4. Сравнение ALL/ANY с подзапросом без фильтра на NULL.

```
SELECT name FROM w_country
WHERE population > ALL (SELECT population FROM w_country
WHERE continent = 'Europe'); -- может дать UNKNOWN из-за NULL
-- Решение
WHERE population > ALL (
    SELECT population FROM w_country WHERE continent = 'Europe'
AND population IS NOT NULL
);
```

5. Ожидание упорядоченности внутри подзапроса.

```
-- ORDER BY в подзапросе без LIMIT не гарантирует порядок
во внешнем запросе
SELECT *
FROM (
    SELECT name FROM w_city ORDER BY population DESC
) t; -- порядок не гарантирован
```


Общие табличные выражения

В PostgreSQL конструкция Common Table Expressions (CTE), задаваемая с помощью оператора **WITH**, представляет собой механизм определения временных именованных результирующих наборов, доступных в рамках одного SQL-запроса. CTE позволяют структурировать сложные запросы, повышать их читаемость и поддерживаемость, а также реализовывать рекурсивные вычисления.

Общая форма записи представлена ниже.

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT ...  
FROM cte_name;
```

CTE логически функционирует как временное представление, существующее только в пределах выполняемого запроса.

МАТЕРИАЛИЗАЦИЯ CTE: ИСТОРИЧЕСКОЕ ПОВЕДЕНИЕ

До версии 12 PostgreSQL применял обязательную материализацию CTE. Это означало следующее.

1. Подзапрос внутри CTE полностью выполнялся.
2. Его результат сохранялся во временной структуре (в памяти или на диске).
3. Основной запрос обращался уже к сохранённому результату.

Таким образом, CTE представлял собой оптимизационный барьер: планировщик запросов не мог «протолкнуть» условия из внешнего запроса внутрь CTE.

ПРИМЕР

```
WITH big_cte AS (  
    SELECT * FROM orders WHERE amount > 1000  
)  
SELECT *  
FROM big_cte  
WHERE customer_id = 10;
```

При обязательной материализации сначала формируется полный набор строк, удовлетворяющих условию `amount > 1000`. Если таких строк, например, один миллион, весь этот набор будет создан и сохранён. Лишь после этого применяется фильтрация `customer_id = 10`.

Даже если итоговый результат содержит всего несколько строк, промежуточный набор формируется полностью, что может приводить к значительным издержкам по времени и памяти.

INLINE-ОПТИМИЗАЦИЯ (ВСТРАИВАНИЕ)

Начиная с версии 12 PostgreSQL изменил стратегию обработки CTE. Теперь планировщик может выполнять inline-оптимизацию (встраивание) CTE в основной запрос.

Встраивание означает, что CTE не рассматривается как изолированный этап выполнения, а логически подставляется внутрь основного запроса. В приведённом примере это эквивалентно преобразованию запроса к форме.

```
SELECT *  
FROM orders  
WHERE amount > 1000  
       AND customer_id = 10;
```

В этом случае фильтры объединяются на этапе построения плана, и база данных может использовать индексы и другие механизмы оптимизации более эффективно. Промежуточный набор из миллиона строк не формируется.

УПРАВЛЕНИЕ СТРАТЕГИЕЙ ВЫПОЛНЕНИЯ

Начиная с версии 12, поведение CTE можно явно контролировать.

```
WITH cte_name AS MATERIALIZED (...)  
WITH cte_name AS NOT MATERIALIZED (...)
```

- **MATERIALIZED** принудительно создаёт промежуточный результат.
- **NOT MATERIALIZED** позволяет планировщику встроить CTE в основной запрос.

По умолчанию PostgreSQL самостоятельно принимает решение, основываясь на оценке стоимости выполнения.

ПРИМЕНЕНИЕ

В современных версиях PostgreSQL CTE и подзапросы в большинстве случаев демонстрируют сопоставимую производительность. Выбор между ними определяется:

- удобством чтения и сопровождения кода,
- необходимостью рекурсии,
- потребностью в явной фиксации промежуточного результата.

При использовании нескольких CTE в одном запросе общие табличные выражения необходимо объявлять последовательно через запятую.

```
WITH first_CTE AS (  
    SELECT person_id, name  
    FROM employee  
    WHERE head_id IS NULL  
)  
, second_CTE AS (  
    SELECT person_id, name  
    FROM employee  
    WHERE head_id IS NOT NULL  
)  
SELECT * FROM first_CTE  
UNION ALL  
SELECT * FROM second_CTE;
```

Рассмотрим примеры запросов на основе таблицы сотрудников, включающей идентификационный номер работника, имя и идентификационный номер его начальника.

```
DROP TABLE IF EXISTS employee;
```

```
CREATE TABLE employee
```

```
(
```

```
    id int,
```

```
    name text,
```

```
    head_id int
```

```
);
```

```
INSERT INTO employee
```

```
    VALUES
```

```
(1, 'Елена', NULL),
```

```
(2, 'Кирилл', 1),
```

```
(3, 'Андрей', 2),
```

```
(4, 'Анастасия', 1),
```

```
(5, 'Александр', 4),
```

```
(6, 'Константин', 4),
```

```
(7, 'Ольга', 6),
```

```
(8, 'Мария', 1),
```

```
(9, 'Владислав', 7),
```

```
(10, 'Олег', 4)
```

```
;
```

person_id	name	head_id
1	Елена	NULL
2	Кирилл	1
3	Андрей	2
4	Анастасия	1
5	Александр	4
6	Константин	4
7	Ольга	5
8	Мария	1
9	Владислав	7
10	Олег	4

Запрос, который выберет всех сотрудников с их начальниками, будет выглядеть следующим образом.

```
with hierarchy as (
    select person_id, name, head_id
    from employee
);
select * from hierarchy;
```

РЕКУРСИВНЫЕ CTE

Рекурсивные CTE позволяют выполнять итеративные вычисления, например организовывать иерархические структуры. Они состоят из двух частей, которые соединяются оператором **UNION ALL**.

1. Начальный (якорный) запрос формирует начальный набор данных.
2. Рекурсивный запрос ссылается на CTE и выполняется до тех пор, пока не будут обработаны все уровни иерархии.

```
WITH RECURSIVE hierarchy AS (  
    SELECT  
        id,  
        name,  
        head_id,  
        1 AS level  
    FROM employee  
    WHERE head_id IS NULL  
    UNION ALL  
    SELECT  
        e.id,  
        e.name,  
        e.head_id,  
        h.level + 1  
    FROM employee e  
        INNER JOIN hierarchy h  
            ON e.head_id = h.id  
    WHERE h.level < 3      -- ограничение  
        глубины рекурсии  
)  
SELECT *  
FROM hierarchy  
order by 1;
```

Первая часть — якорный запрос, который выбирает начальников, вторая часть — рекурсивный запрос для выбора подчинённых. Также можно заметить, что был использован фильтр **WHERE h.level <= 3**, ограничения глубины рекурсии.

```
WITH RECURSIVE numbers(n) AS (  
    SELECT 1  
    UNION ALL  
    SELECT n + 1  
    FROM numbers  
    WHERE n < 10  
)  
SELECT n FROM numbers;
```

Параметр **n** в CTE **numbers(n)** задаёт имя столбца в результирующем наборе. CTE работает как временная таблица со столбцами, и, если не указать имя, СУБД не поймёт, как обращаться к данным в рекурсивной части.

КАК РАБОТАЕТ МЕХАНИЗМ

1. Анкерная часть создаёт первую строку: **{n: 1}**
2. Рекурсивная часть читает столбец **n** из текущего результата
3. Создаёт новую строку: **{n: 2}** → добавляет в конец
4. Повторяет до **WHERE n < 10**

Рекурсия остановится, когда рекурсивная часть вернёт 0 строк.

Заключение

Подзапросы делают фильтрацию гибкой. Ключевые практики:

- используй **EXISTS/NOT EXISTS** для проверки наличия/отсутствия (надёжно при **NULL**);
- для **IN/NOT IN** фильтруй **NULL** в подзапросах;
- явно приводи типы при сравнении разнородных колонок;
- помни про особенности **ANY/ALL** и трёхзначную логику SQL.

CTE (WITH):

- применяй CTE для структурирования сложных запросов и повышения читаемости;
- используй **WITH RECURSIVE** для иерархий;
- учитывай, что начиная с PostgreSQL 12 CTE могут быть inline-оптимизированы и не всегда материализуются;
- при необходимости явно управляй стратегией выполнения (**MATERIALIZED / NOT MATERIALIZED**).